Hosting a Next.js Project on Vercel

A complete step-by-step deployment guide

This guide covers two methods — the Vercel dashboard (easiest) and the Vercel CLI (for automation). It also explains how to connect a custom domain and set environment variables.

1. Prerequisites

Before you start, make sure you have:

- A Next.js project that builds locally with `npm run build`.
- A GitHub/GitLab/Bitbucket repository with your code pushed to it.
- A free Vercel account (sign up at https://vercel.com/signup).
- Node.js 18.17+ installed on your machine (for the CLI method).
- A custom domain purchased from a registrar (optional, for step 5).

2. Method A — Deploy via the Vercel Dashboard (easiest)

This is the recommended way for most users. No local tools required.

Step 1: Import your repository

- Go to https://vercel.com/new.
- Under “Import Git Repository”, find your repo.
- If it is not listed, click “Adjust GitHub App Permissions” and grant access.
- Click “Import”.

Step 2: Configure the project

Vercel auto-detects Next.js from package.json and next.config. Leave the defaults:

- Framework Preset: Next.js
- Build Command: next build (auto-filled)
- Output Directory: .next (auto-filled)
- Install Command: npm install (auto-filled)

Click “Deploy”. The first build finishes in 2–3 minutes.

Step 3: Watch the build

Vercel runs `npm install` then `next build`. When it finishes you get a live URL like:

```
https://your-project-name.vercel.app
```

! If the build fails on Vercel but passes locally, the most common cause is a peer-dependency

conflict (ERESOLVE) on a fresh install. Add a file named .npmrc in your project root with the line: legacy-peer-deps=true — then commit and redeploy.

! If the build fails on Vercel but passes locally, the most common cause is a peer-dependency

conflict (ERESOLVE) on a fresh install. Add a file named .npmrc in your project root with the line: legacy-peer-deps=true — then commit and redeploy.

3. Method B — Deploy via the Vercel CLI

Use this when you want to deploy from your terminal or automate deploys.

Step 1: Install the Vercel CLI

```
npm i -g vercel
```

Step 2: Log in

```
vercel login
```

Follow the prompts (email or GitHub). This authenticates your machine.

Step 3: Deploy a preview

From your project root, run:

```
vercel
```

Answer the prompts (set up and deploy, confirm scope, project name, directory). This creates a preview deployment at a unique *.vercel.app URL.

Step 4: Deploy to production

```
vercel --prod
```

This promotes the build to your production URL.

! To skip the interactive prompts, use: vercel --prod --yes. If the auto-generated project name is rejected, the CLI will tell you — just retry.

4. Set Environment Variables

Environment variables are NOT copied from your .env.local. You must add them in Vercel.

Via the dashboard

- Go to Project !’ Settings !’ Environment Variables.
- Add each variable (name + value).
- Choose the environment: Production, Preview, or Development.
- Click “Redeploy” so the new build picks them up.

Via the CLI

Add a variable (you will be prompted for the value):

```
```

```
vercel env add VARIABLE_NAME production
```

Pull variables locally to .env.local:

```
vercel env pull .env.local
```

Remove a variable:

```
vercel env rm VARIABLE_NAME production --yes
```

! Never commit real secrets to git. Add them only in the Vercel dashboard or via the CLI. Keep .env.local in .gitignore.

5. Connect a Custom Domain

Step 1: Add the domain in Vercel

- Go to Project !' Settings !' Domains.
- Click “Add”, enter your domain (e.g. mysite.com), click “Add”.
- Add the www variant (www.mysite.com) and set it to redirect to the root.

Step 2: Create DNS records at your registrar

Vercel shows you the exact records. Typically:

```
Type     Host     Value / Points to
A        @        76.76.21.21
CNAME    www      cname.vercel-dns.com
```

Step 3: Verify

- Wait for the DNS + Domain Configuration checks to turn green (minutes to 48h).
- Vercel auto-provisions an SSL/TLS certificate via Let's Encrypt.
- Once it says “Valid Certificate”, your site is live at https://mysite.com.

Step 4: Update NEXT_PUBLIC_SITE_URL

If your app uses the site URL for metadata/sitemap/OG tags, set NEXT_PUBLIC_SITE_URL to your final domain (https://mysite.com) and redeploy.

6. Automatic Deploys from Git

Once your repo is connected, every `git push` to your main branch triggers a new production deployment. Pull requests get isolated Preview deployments with their own URLs.

```
git add .
git commit -m "feat: update site"
git push origin main   # triggers a production deploy
```

7. Common Issues & Fixes

Build fails: npm install exits with code 1 (ERESOLVE)

Peer-dependency conflict on a fresh install. Fix:

```
echo legacy-peer-deps=true > .npmrc
```

Commit it and redeploy.

Build fails: Node version mismatch

Set the Node version in Project !’ Settings !’ General !’ Node.js Version (18.x, 20.x, or 22.x). Match your local version.

Domain stays “Pending”

Double-check the A/CNAME records at your registrar. DNS propagation can take up to 48 hours. Lower the TTL to speed it up.

Environment variable not taking effect

Environment variables are baked in at build time. After adding or changing one, you must Redeploy — just changing the value does not update the live site.

404 on dynamic routes after deploy

Make sure the required environment variables are present for the Production environment, then redeploy.

8. Summary Checklist

- Project builds locally with `npm run build`.
- Code pushed to a Git repository.
- Imported into Vercel (dashboard or CLI).
- First deployment is live at *.vercel.app.
- Environment variables added and redeployed.
- Custom domain added in Settings !’ Domains.
- DNS A + CNAME records created at registrar.
- SSL certificate provisioned (green checkmark).
- NEXT_PUBLIC_SITE_URL set to final domain + redeploy.

That’s it — your Next.js project is now live on Vercel with your custom domain. Every future push to your main branch redeploys automatically.